第五章作业：

1. 定义一个Cat类，拥有静态数据成员numOfCats，记录Cat的个体数目；静态成员函数getNumOfCats()，存取numOfCats。设计程序测试这个类，体会静态数据成员和静态成员函数的用法。

```
#include <iostream>
using namespace std;
class cat
{
public:
	cat(){ numofcats++; }
	static int getnumofcats() { return numofcats; }//静态成员函数
	~cat() { numofcats--; }
private:
	static int numofcats;//静态数据成员，cat个数
};
int cat::numofcats=0;//数量初始化为0
int main()
{
	cat a;
	cat b;
	cout<<"总共有"<<cat::getnumofcats()<<"只猫"<<endl;
	return 0;
}
```

```
总共有2辆车
Program ended with exit code: 0
```

2.声明Boat与Car两个类，二者都有weight属性，定义二者的一个友元函数totalWeight()，计算二者的重量和。//没有说清函数参数getTotalWeight(Car &aCar, Boat &aBoat)

```
#include <iostream>
using namespace std;
class boat;
class car
{
public:
	car(int weight):weight(weight){}
	~car() {}
	friend int getTotalWeight(car &car, boat &boat);
private:
	int weight;
};
class boat{
public:
	boat(int weight):weight(weight){}
	~boat(){}
	friend int getTotalWeight(car &car, boat &boat);
private:
	int weight;
};
int getTotalWeight(car &car, boat &boat)
{
	return car.weight+boat.weight;
};
int main()
{
	car a(10);
	boat b(20);
	cout<<"总重
"<<getTotalWeight(a,b)<<"kg"<<endl;
	return 0;
}
```

```
总重30kg
Program ended with exit code: 0
```

第六章作业：

1.const int *p1 和 int *const p2 的区别是什么？

const char * p1：定义一个指向字符（串）常量的指针；char * const p2：定义一个指向字符（串）的指针常量

2.编写一个矩阵转置的函数 void transpose(int *matrix,int m,int n)，矩阵的行、列数在程序中由用户动态输入。

提示：(a).本题要求原地转置（转置后数据还放在原空间）《因为形参中没有目标空间》

(b).int *matrix 提示你只能用一维数组

(c).如果没有（a）限制，你能否写出：

void transpose2(int **matrix,int **Tmatrix,int m,int n);《类似指针数组》

void transpose3(int (*matrix)[n],int (*Tmatrix)[m],int m,int n);《二维数组、数组指针》

```
#include<iostream>
using namespace std;
void transpose(int *matrix, int m, int n)
{
    for(int i=0; i<m*n; i++)
    {
        int next = (i%n)*m + i/n;
        while(next > i)
            next = (next%n)*m + next/n;
        if(next == i)
        {
            int t = matrix[i];
            int cur = i;
            int pre = (i%m)*n + i/m;
            while(pre != i)
            {
                matrix[cur] = matrix[pre];
                cur = pre;
                pre = (cur%m)*n + cur/m;
            }
            matrix[cur] = t;
        }
    }
}
void print(int *matrix, int m, int n)
{
    for(int i=0; i<m*n; i++)
    {
        if((i+1)%n == 0)
            cout << matrix[i] << "\n";
        else
            cout << matrix[i] << " ";
    }
}
int main()
{
    int i,m,c,l;
    cout<<"输出原矩阵的行、列各为： ";
    cin>>i>>m;
    cout<<"输出转置后矩阵的行、列各为： ";
    cin>>c>>l;
    cout<<"输入原矩阵中的元素";
    int matrix[i*m];
    for(int k=0;k<i*m;k++)
        cin>>matrix[k];
    cout << "Before:" << endl;
    print(matrix, i, m);
    transpose(matrix, i, m);
    cout << "After:" << endl;
    print(matrix, c, l);
    return 0;
}
```

```
输出原矩阵的行、列各为: 4 2
输出转置后矩阵的行、列各为: 2 4
输入原矩阵中的元素1 2 3 4 5 6 7 8
Before:
1 2
3 4
5 6
7 8
After:
1 3 5 7
2 4 6 8
Program ended with exit code: 0
```

期中自测

二、填空题（1×10=10 分）

1、C++中的函数名是一个__指向函数的__指针，该指针指向__该函数所在内存区域的首__地址。

2、C++语言中的数组名是___指向该数组的__指针，该指针指向__该数组第一个元素的__地址。

3、一组同名的重载函数可以根据__参数个数___、__参数类型___或__形式参数__进行区分。

4、定义一个指向字符（串）常量的指针的语句是__const char * p__；定义一个指向字符（串）的指针常量的语句是__char * const p__。

5、在 C++语言中，类的非静态成员函数中有一个隐含的 this 指针，该指针指向__调用该函数的对象__。

四、编程题（共 40 分）

```
class Circle{
public:

private:
    char *pCir_Name;      //指向圆名称字符串
    int x,y;   //圆心坐标
    int radius;     //圆的半径
};
```

1、描述圆的类 Circle 部分代码如下，按要求添加成员函数完善程序：

（1）、添加静态数据成员 cir_Count 描述程序运行时圆对象总数并进行初始化、定义相应的静态成员函数获取该数据；

（2）、定义带参数的构造函数初始化数据成员；

（3）、定义拷贝构造函数（深拷贝）与析构函数（释放动态申请的内存单元）；

（4）、定义成员函数获取圆的名称；

（5）、写一个主程序测试上述功能。（10 分）

```
#include <iostream>
using namespace std;
class circle{
public:
    circle(int X,int Y,int Radius,char *PCir_Name){
        x=X;
        y=Y;
        radius=Radius;
        if(PCir_Name==NULL){
            pCir_Name = new char[1];
```

```
            *pCir_Name = '\0';
            cout<<"1";
        }
        else{
            pCir_Name = new
char[strlen(PCir_Name)+1];
            strcpy(pCir_Name,PCir_Name);
        }
        cout<<"数据拷贝完毕！"<<endl;
    }
    ~circle()
    {
        delete [] pCir_Name;
        cout<<"name数据删除完毕！"<<endl;
    }
    void setname(char *PCir_Name)
    {
        delete [] pCir_Name;
        pCir_Name = new
char[strlen(PCir_Name)+1];
            strcpy(pCir_Name, PCir_Name);
    }
    char * const getpCir_Name(){ return
pCir_Name;}
private:
    char *pCir_Name;        //指向圆名称字符串
    int x,y;                //圆心坐标
    int radius;             //圆的半径
    static int cir_count;   //静态数据成员
};
int main(int argc, const char * argv[]) {
    circle a(1,1,10,"cir1");
    circle b(2,2,20,"cir2");
    cout<<a.getpCir_Name()<<endl;
    cout<<b.getpCir_Name()<<endl;
    return 0;
}
```

```
数据拷贝完毕!
数据拷贝完毕!
cir1
cir2
name数据删除完毕!
name数据删除完毕!
Program ended with exit code: 0
```

2、 定义一个点类 Point，要求如下：

（1）、定义相应的数据成员描述点的坐标（int 类型）；

（2）、定义带参数的构造函数初始化数据成员；

（3）、定义友元函数 float Distance(Point &p1,Point &p2)用于计算两个点之间的距离；

（4）、写一个主程序测试上述功能。（10 分）

```
#include <iostream>
#include <math.h>
using namespace std;
class point{
public:
    point(int X,int Y):x(X),y(Y) {}
    ~point() {}
    friend float distanse(point &p1,point &p2){
        return sqrt((p1.x-p2.x)*(p1.x-p2.x)+(p1.y-p2.y)*(p1.y-p2.y));
    };
```

```
p1和p2的两点坐标距离为: 5
Program ended with exit code: 0
```

```
private:
    int x;
    int y;
};
int main(int argc, const char * argv[]) {
    point p1(0,0);
    point p2(3,4);
    cout<< "p1和p2的两点坐标距离为：  "<<distanse(p1,p2)<<endl;
    return 0;
}
```

3、描述整型数组类 MyIntArray 部分代码如下，按要求添加成员函数完善程序：

（1）、MyIntArray mia（10）；该语句定义的对象 mia 代表包含 10 个元素的整型数组；

（2）、MyIntArray mib=mia；用 mia 对象初始化 mib 对象；

（3）、cout<<mia；该语句能够输出数组的所有元素；

（4）、mia.init();该函数将数组元素初始化为 1-100 之间的随机整数；

（5）、写一个主程序测试上述功能。（20 分）

```
class MyIntArray{
public:

private:
    int *pInt;      //整型数组首地址
    int length;     //数组元素个数
};
```

```
#include <iostream>
#include <time.h>
#include <stdlib.h>
using namespace std;
class MyIntArray{
public:
    MyIntArray(){
        length=1;
        pInt = new int(0);
    }
    MyIntArray(int size){
        pInt = new int[size];
        length=size;
    }
    MyIntArray(MyIntArray &array){
        length=array.length;
        pInt = new int[length];
        for(int i=0;i<length;i++)
            pInt[i]=array.pInt[i];
    }//按mib大小申请空间，循环拷数据
    ~MyIntArray(){
        if(pInt!=NULL)
            delete[] pInt;
    }
    void init(){
        srand((unsigned)time(NULL));
        for(int i=0;i<length;i++)
            pInt[i]=rand()%100+1;//1到100随机数
    }
    friend ostream &operator<<(ostream
&out,const MyIntArray &array);
    int & operator[](int index) {
      if (index >= 0 && index < length)
          return pInt[index];
      else
          throw "数组下标越界" ;
    }
private:
    int *pInt;
    int length;
};
ostream &operator<<(ostream &out,const
MyIntArray &array)
{
        out<<"array data:"<<endl;
        for(int i=0;i<array.length;i++)
            out<<array.pInt[i]<<" ";
        out<<endl;
        return out;
}
```

```
int main()
{
    MyIntArray mia(10);
    mia.init();
    MyIntArray mib(mia);
    cout<<mia<<endl;
    cout<<mib<<endl;
    return 0;
}
```

```
array data:
11 1 60 65 77 70 17 91 49 23

array data:
11 1 60 65 77 70 17 91 49 23

Program ended with exit code: 0
```